Compiling Typed APIs into Context-Free Languages

Breandan Considine, Clemente Pasti, Tim Vieira

August 13, 2026

Motivation: type-safe code completion

- Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- For any $\sigma \in \Sigma^*$ such that $\exists \tau.\sigma\tau \in \ell_\Gamma$, show τ's in shortlex order

Motivation: type-safe code completion

- Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- For any $\sigma \in \Sigma^*$ such that $\exists \tau . \sigma\tau \in \ell_\Gamma$, show τ's in shortlex order

```
#include <memory>
long* codes(); double* groups();
const char* text(); float* samples();
std::unique_ptr<int> fresh(), parse(const char*);
std::unique_ptr<int> average(float*), select(double&);
std::unique_ptr<int> combine(const char*, long*);
std::unique_ptr<int> schedule(const char*, long*, float*);

int main(int argc, char** args) {
  std::unique_ptr<int> b
}
```

Motivation: type-safe code completion

- Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- For any $\sigma \in \Sigma^*$ such that $\exists\tau.\sigma\tau \in \ell_\Gamma$, show τ's in shortlex order

```
1  #include <memory>
2  long* codes(); double* groups();
3  const char* text(); float* samples();
4  std::unique_ptr<int> fresh(), parse(const char*);
5  std::unique_ptr<int> average(float*), select(double&);
6  std::unique_ptr<int> combine(const char*, long*);
7  std::unique_ptr<int> schedule(const char*, long*, float*);
8
9  int main(int argc, char** args) {
10   std::unique_ptr<int> b
11 }
```

```
;
= fresh();
= average(samples());
= parse(text());
= select(groups()[argc]);
= combine(text(), codes());
= schedule(text(), codes(), samples());
```

Motivation: type-safe code completion

- Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- For any $\sigma \in \Sigma^*$ such that $\exists\tau.\sigma\tau \in \ell_\Gamma$, show τ's in shortlex order

```
1  #include <memory>
2  long* codes(); double* groups();
3  const char* text(); float* samples();
4  std::unique_ptr<int> fresh(), parse(const char*);
5  std::unique_ptr<int> average(float*), select(double&);
6  std::unique_ptr<int> combine(const char*, long*);
7  std::unique_ptr<int> schedule(const char*, long*, float*);
8
9  int main(int argc, char** args) {
10   std::unique_ptr<int> b
11 }
```

```
;
= fresh();
= average(samples());
= parse(text());
= select(groups()[argc]);
= combine(text(), codes());
= schedule(text(), codes(), samples());
```

? Why shortlex order?

- Parsimonious abduction
- Bounded-delay enumeration
- Efficient rank and unrank queries

Prior work on type-safe code completion

Legend ✓ by construction **gen** generation-time **filt** post-generation ∼ partial / empirical – absent / unclaimed

Category
Literature
Language
Output unit
Query format
Core theory
Ranking
Learning
Filter timing
Syntax safe
Type safe
Statements
Parametricity
Usable tool
Public URL

Prior work on type-safe code completion

Legend ✓ by construction **gen** generation-time **filt** post-generation ∼ partial / empirical – absent / unclaimed

Category	**Type-directed search**		
Literature	**Perelman'12**	**Gvero'13**	**Pelsmaeker'22**
Language	C#	Scala	Statix DSL
Output unit	expression	expression	fragment
Query format	partial expr	typed hole	cursor
Core theory	bespoke	STLC	SDF3/Statix
Ranking	type distance	usage priors	solver order
Learning	–	∼	–
Filter timing	search	search	solver
Syntax safe	✓	✓	✓
Type safe	✓	✓	✓
Statements	–	–	∼
Parametricity	–	–	✓
Usable tool	VS Code	Eclipse	Spoofax
Public URL	–	code	artifact

Prior work on type-safe code completion

Legend ✓ by construction **gen** generation-time **filt** post-generation ∼ partial / empirical – absent / unclaimed

Category	**Type-directed search**			**Learned completion**	
Literature	**Perelman'12**	**Gvero'13**	**Pelsmaeker'22**	**Raychev'14**	**Semenkin'24**
Language	C#	Scala	Statix DSL	Java	Py+JVM+JS
Output unit	expression	expression	fragment	call sequence	full line
Query format	partial expr	typed hole	cursor	holes	left prefix
Core theory	bespoke	STLC	SDF3/Statix	bespoke	–
Ranking	type distance	usage priors	solver order	RNN	LLM
Learning	–	∼	–	✓	✓
Filter timing	search	search	solver	rank only	**filt**
Syntax safe	✓	✓	✓	∼	**filt**
Type safe	✓	✓	✓	∼	–
Statements	–	–	∼	∼	✓
Parametricity	–	–	✓	–	–
Usable tool	VS Code	Eclipse	Spoofax	Slang	JetBrains
Public URL	–	code	artifact	–	plugin

Prior work on type-safe code completion

Legend ✓ by construction **gen** generation-time **filt** post-generation ∼ partial / empirical – absent / unclaimed

Category	**Type-directed search**			**Learned completion**		**Constrained**
Literature	**Perelman'12**	**Gvero'13**	**Pelsmaeker'22**	**Raychev'14**	**Semenkin'24**	**Mündler'25**
Language	C#	Scala	Statix DSL	Java	Py+JVM+JS	TypeScript
Output unit	expression	expression	fragment	call sequence	full line	function
Query format	partial expr	typed hole	cursor	holes	left prefix	prompt
Core theory	bespoke	STLC	SDF3/Statix	bespoke	–	prefix FSA
Ranking	type distance	usage priors	solver order	RNN	LLM	LLM
Learning	–	∼	–	✓	✓	✓
Filter timing	search	search	solver	rank only	**filt**	**gen**
Syntax safe	✓	✓	✓	∼	**filt**	**gen**
Type safe	✓	✓	✓	∼	–	**gen**
Statements	–	–	∼	∼	✓	✓
Parametricity	–	–	✓	–	–	∼
Usable tool	VS Code	Eclipse	Spoofax	Slang	JetBrains	Git repo
Public URL	–	code	artifact	–	plugin	code

Prior work on type-safe code completion

Legend ✓ by construction **gen** generation-time **filt** post-generation ∼ partial / empirical – absent / unclaimed

Category	**Type-directed search**			**Learned completion**		**Constrained**
Literature	**Perelman'12**	**Gvero'13**	**Pelsmaeker'22**	**Raychev'14**	**Semenkin'24**	**Mündler'25**
Language	C#	Scala	Statix DSL	Java	Py+JVM+JS	TypeScript
Output unit	expression	expression	fragment	call sequence	full line	function
Query format	partial expr	typed hole	cursor	holes	left prefix	prompt
Core theory	bespoke	STLC	SDF3/Statix	bespoke	–	prefix FSA
Ranking	type distance	usage priors	solver order	RNN	LLM	LLM
Learning	–	∼	–	✓	✓	✓
Filter timing	search	search	solver	rank only	**filt**	**gen**
Syntax safe	✓	✓	✓	∼	**filt**	**gen**
Type safe	✓	✓	✓	∼	–	**gen**
Statements	–	–	∼	∼	✓	✓
Parametricity	–	–	✓	–	–	∼
Usable tool	VS Code	Eclipse	Spoofax	Slang	JetBrains	Git repo
Public URL	–	code	artifact	–	plugin	code

Gap Prior work tends to trade off probabilistic ranking, completion length, and type safety but does not guarantee bounded-delay enumeration for all and only well-typed statements.

How long are typical source code statements?

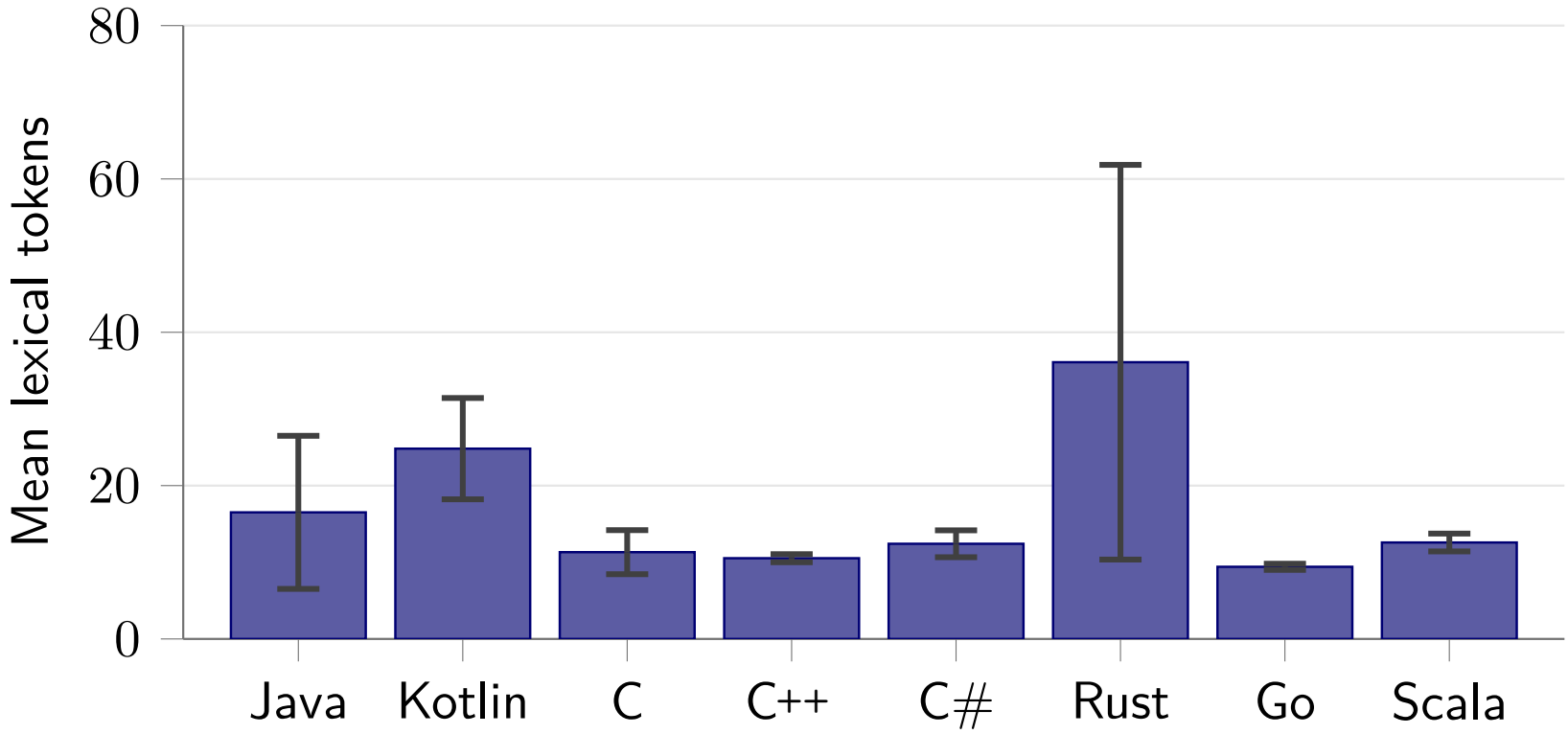


Figure: Whiskers are 95% CIs over sampled files. Statements are Tree-sitter statement nodes with comments and whitespace ignored. Samples are random GitHub files > 20kb; per-language sample sizes are shown below.

How large are programming language grammars?

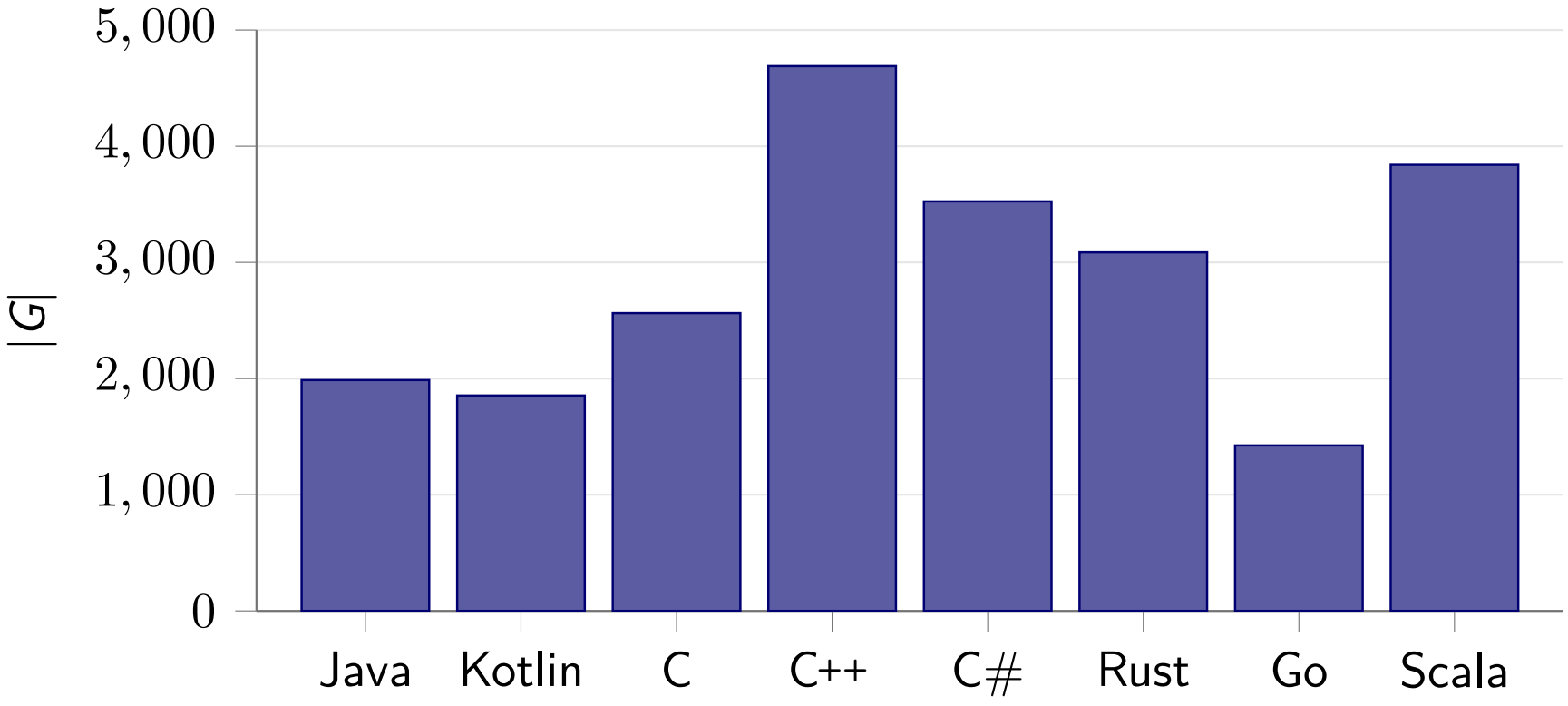


Figure: C++ is a syntactically feature-rich language. Grammar size calculated from Tree-sitter reference CFGs, where $|G| := \sum_{\substack{w \in V \\ (w \to \sigma) \in P}} |w\sigma|$.

Correctness criterion: what must the CFG preserve?

Surface fragment G' e.g., names, invocation, member chains

$S ::= E;\quad E ::= \texttt{ID} \mid \texttt{ID()} \mid \texttt{ID(}\,\texttt{ARGS}\,\texttt{)} \mid E.E\ \texttt{ARGS} ::= E \mid E, \texttt{ARGS}$

Correctness criterion: what must the CFG preserve?

Surface fragment G' e.g., names, invocation, member chains

$S ::= E;\quad E ::= \texttt{ID} \mid \texttt{ID()} \mid \texttt{ID(ARGS)} \mid E.E\quad \texttt{ARGS} ::= E \mid E, \texttt{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$$\pi_\Sigma(\texttt{x.f(y)}) = \texttt{ID.ID(ID)}$$

checks that the shape belongs to G_0

Correctness criterion: what must the CFG preserve?

Surface fragment G' e.g., names, invocation, member chains

$S ::= E;\quad E ::= \texttt{ID} \mid \texttt{ID()} \mid \texttt{ID(ARGS)} \mid E.E\quad \texttt{ARGS} ::= E \mid E, \texttt{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$\pi_\Sigma(\texttt{x.f(y)}) = \texttt{ID.ID(ID)}$

checks that the shape belongs to G_0

2. Typed projection Π_Γ

names $\rightarrow$ type slots; API names stay

$\Pi_\Gamma(\texttt{x.f(y)}) \ni \texttt{E}\langle X\rangle\texttt{.f(E}\langle Y\rangle\texttt{)}$

contains all and only well-typed terms

Correctness criterion: what must the CFG preserve?

Surface fragment G' e.g., names, invocation, member chains

$S ::= E;\quad E ::= \texttt{ID} \mid \texttt{ID()} \mid \texttt{ID(ARGS)} \mid E.E\quad \texttt{ARGS} ::= E \mid E, \texttt{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$$\pi_\Sigma(\texttt{x.f(y)}) = \texttt{ID.ID(ID)}$$

checks that the shape belongs to G_0

2. Typed projection Π_Γ

names $\to$ type slots; API names stay

$$\Pi_\Gamma(\texttt{x.f(y)}) \ni \texttt{E}\langle X\rangle\texttt{.f(E}\langle Y\rangle\texttt{)}$$

contains all and only well-typed terms

Correctness criterion

For a target language L with surface fragment G', construct G_Γ satisfying

$$\pi_\Sigma(\sigma) \in \mathcal{L}(G') \quad \Longleftrightarrow \quad \Gamma \vdash \Pi_\Gamma(\sigma) \subset \mathcal{L}(G_\Gamma)$$

for all and only source statements σ accepted by a reference compiler for L.

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\text{C++}} \cup$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

$G_{\texttt{com/buzz.hpp}} \cup$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

$G_{\texttt{com/buzz.hpp}} \cup$

$\{ \text{Type} \rightarrow \texttt{string} \} \cup$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

$G_{\texttt{com/buzz.hpp}} \cup$

$\{\,\text{Type} \rightarrow \texttt{string}\,\} \cup$

$\ldots$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

$G_{\texttt{com/buzz.hpp}} \cup$

$\{\, \text{Type} \rightarrow \texttt{string} \,\} \cup$

$\ldots$

$\{\, \text{Int} \rightarrow \texttt{fizz()},\ \text{S} \rightarrow \texttt{fizz();} \,\} \cup$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

$G_{\texttt{com/buzz.hpp}} \cup$

$\{\, \text{Type} \rightarrow \texttt{string} \,\} \cup$

$\ldots$

$\{\, \text{Int} \rightarrow \texttt{fizz()},\ \text{S} \rightarrow \texttt{fizz();} \,\} \cup$

$\{\, \text{Str} \rightarrow \texttt{buzz(}\ \text{Str}\ \texttt{)},\ \text{S} \rightarrow \texttt{buzz(}\ \text{Str}\ \texttt{);} \,\} \cup$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

$G_{\texttt{com/buzz.hpp}} \cup$

$\{\, \text{Type} \rightarrow \texttt{string} \,\} \cup$

$\ldots$

$\{\, \text{Int} \rightarrow \texttt{fizz()},\ \text{S} \rightarrow \texttt{fizz();} \,\} \cup$

$\{\, \text{Str} \rightarrow \texttt{buzz(}\ \text{Str}\ \texttt{)},\ \text{S} \rightarrow \texttt{buzz(}\ \text{Str}\ \texttt{);} \,\} \cup$

$\ldots$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

$G_{\texttt{com/buzz.hpp}} \cup$

$\{\, \text{Type} \rightarrow \texttt{string} \,\} \cup$

$\ldots$

$\{\, \text{Int} \rightarrow \texttt{fizz()},\ \text{S} \rightarrow \texttt{fizz();} \,\} \cup$

$\{\, \text{Str} \rightarrow \texttt{buzz(}\ \text{Str}\ \texttt{)},\ \text{S} \rightarrow \texttt{buzz(}\ \text{Str}\ \texttt{);} \,\} \cup$

$\ldots$

$\{\, \text{Int} \rightarrow \texttt{argc},\ \text{CharPtr} \rightarrow \texttt{args[}\ \text{Int}\ \texttt{]} \,\} \cup$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   □
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

$G_{\texttt{com/buzz.hpp}} \cup$

$\{\, \text{Type} \to \texttt{string} \,\} \cup$

$\ldots$

$\{\, \text{Int} \to \texttt{fizz()},\ \text{S} \to \texttt{fizz();} \,\} \cup$

$\{\, \text{Str} \to \texttt{buzz(}\ \text{Str}\ \texttt{)},\ \text{S} \to \texttt{buzz(}\ \text{Str}\ \texttt{);} \,\} \cup$

$\ldots$

$\{\, \text{Int} \to \texttt{argc},\ \text{CharPtr} \to \texttt{args[}\ \text{Int}\ \texttt{]} \,\} \cup$

$\{\, \text{Str} \to \texttt{bang} \,\} \to G_{\Gamma}$

Grammar modularization

C++ context

```
1  // Filetype: *.cpp
2  #include <string>
3  #include <vector>
4  #include "org/fizz.hpp"
5  #include "com/buzz.hpp"
6  using std::string;
7  // ...
8  int fizz() { return 3; }
9  string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12   string bang = "...";
13   
14 }
```

Grammar contribution

$G_{\texttt{C++}} \cup$

$G_{\texttt{<string>}} \cup$

$G_{\texttt{<vector>}} \cup$

$G_{\texttt{org/fizz.hpp}} \cup$

$G_{\texttt{com/buzz.hpp}} \cup$

$\{\,\text{Type} \to \texttt{string}\,\} \cup$

$\ldots$

$\{\,\text{Int} \to \texttt{fizz()},\ \text{S} \to \texttt{fizz();}\,\} \cup$

$\{\,\text{Str} \to \texttt{buzz(}\ \text{Str}\ \texttt{)},\ \text{S} \to \texttt{buzz(}\ \text{Str}\ \texttt{);}\,\} \cup$

$\ldots$

$\{\,\text{Int} \to \texttt{argc},\ \text{CharPtr} \to \texttt{args[}\ \text{Int}\ \texttt{]}\,\} \cup$

$\{\,\text{Str} \to \texttt{bang}\,\} \to G_\Gamma$

$\Gamma \vdash \mathcal{L}(G_\Gamma)$

Nominal typing judgements

Context Γ contains all declarations visible at the cursor

$\Gamma \vdash e : T$ C++ accepts e and gives it the concrete type T.

C++ program

```
struct Meter {
  int read(int n) {
    return n + 1;
  }
};
int main() {
  Meter m;
  int n = 2;
  int x = m.read(n);
  return x + x;
}
```

Embedding

$$\Gamma \vdash \texttt{m.read(n)} : \texttt{int}$$

$$\Downarrow$$

$$\texttt{E}\langle\texttt{int}\rangle \to \texttt{E}\langle\texttt{Meter}\rangle\texttt{.read(E}\langle\texttt{int}\rangle\texttt{)}$$

$$\Gamma \vdash \texttt{x} + \texttt{x} : \texttt{int}$$

$$\Downarrow$$

$$\texttt{E}\langle\texttt{int}\rangle \to \texttt{E}\langle\texttt{int}\rangle\ \texttt{+}\ \texttt{E}\langle\texttt{int}\rangle$$

Embedding Substitute locally bound names with typed slots.

Subtyping rules

Derived-to-base call

$\texttt{Dog} <: \texttt{Animal}$

`Dog` publicly inherits `Animal`, so `d` may be passed to `age` as an `Animal`.

C++ program

```
struct Animal {};
struct Dog : Animal {};
int age(Animal) { return 7; }
int main() {
  Dog d;
  int x = age(d);
  return x;
}
```

Embedding

$$\Gamma \vdash \texttt{age} : \texttt{Animal} \to \texttt{int}$$

$$\Downarrow$$

$$\texttt{E}\langle\texttt{int}\rangle \to \texttt{age(E}\langle\texttt{Dog}\rangle\texttt{)}$$

$$\texttt{E}\langle\texttt{int}\rangle \to \texttt{age(E}\langle\texttt{Animal}\rangle\texttt{)}$$

Scoped subtype inclusion

Only track well-scoped subtypes and embed each subtype monomorphically.

Overload resolution

Ambiguous candidates

```
struct A {}; struct B {};
struct D : A, B {};
int f(A); int f(B);
D d; A a; B b;
int n = f(a);
int m = f(b);
int o = f(d); // ambiguous
```

C++ resolves whole calls

$\texttt{f}(\texttt{A}) : \texttt{D} \to \texttt{A} \qquad \texttt{f}(\texttt{B}) : \texttt{D} \to \texttt{B}$

Both candidates are viable by conversion but unresolvable.

$$\mathrm{resolve}_\Gamma\,(\texttt{f(d)}) = \,?$$

Local subtype closure is unsound

$$\left.\begin{array}{l} \texttt{E}\langle\texttt{int}\rangle \to \texttt{f(E}\langle X\rangle\texttt{)} \\ \texttt{E}\langle X\rangle \to \texttt{E}\langle\texttt{D}\rangle \end{array}\right\} X \in \{\texttt{A}, \texttt{B}\} \Rightarrow_{G_\Gamma} \texttt{E}\langle\texttt{int}\rangle \Rightarrow^* \texttt{f(E}\langle\texttt{D}\rangle\texttt{)}$$

Local subtype expansion derives $\texttt{f(E}\langle\texttt{D}\rangle\texttt{)}$, but C++ rejects `f(d)` as ambiguous.

Resolve before lowering. Emit a ground call production only when the exact call has a unique best viable function; ambiguous calls emit no production.

Covariance and contravariance

Subtype setup

```
struct A {}; struct A_: A {};
struct B {}; struct B_: B {};
B f(A); B_ g(A_);
A a; A_ a_;
```

Input position: contravariant

✓ `f(a_); f(a);`

× `g(a);` Error

An A_ value may be passed to a A parameter but not vis versa.

Result position: covariant

✓ `B  b  = g(a_);`

× `B_ b_ = f(a);` Error

A B_ result may widen to B; a B result cannot narrow to B_.

Template handling

C++ program

```
#include <string>
#include <vector>
using std::string;
template<class T>
T twice(T x) { return x+x; }
int main() {
    string s;
    std::vector<int> d;
    twice(1); // ok
    twice(s); // ok
    twice(d); // + undefined
}
```

Try concrete types

`int`	✓ accepted
`std::string`	✓ accepted
`std::vector<int>`	× rejected

The vector case fails because `x+x` is not defined for vectors.

Filter successful rules

$\Gamma \vdash \texttt{twice(q)} : \texttt{std::string} \Rightarrow \texttt{E}\langle\texttt{std::string}\rangle \rightarrow \texttt{twice(E}\langle\texttt{std::string}\rangle\texttt{)}$

Sound: every kept rule compiles **Complete (in scope):** all scoped types were tried

Template handling: invariant by default

Subtyping under parametricity

$$\texttt{Dog} <: \texttt{Animal} \quad \not\Rightarrow \quad F\langle\texttt{Dog}\rangle <: F\langle\texttt{Animal}\rangle$$

Class templates may optionally provide a constrained converting constructor.

API-provided covariance

```
std::shared_ptr<Dog> pd;
std::shared_ptr<Animal> pa = pd;
// accepted
```

`shared_ptr` explicitly supplies the conversion.

Invariant specialization

```
std::vector<Dog> ds;
std::vector<Animal> as = ds;
// rejected
```

The two `vector` specializations are unrelated.

CFG generation

enumerate ground states; validate exact contexts

Cannot infer that $F\langle D\rangle$ converts to $F\langle B\rangle$ merely from $D <: B$. Emit a rule only when the C++ API supplies and accepts that conversion.

Experimental Design

Dataset & unit

- Standalone `.cpp` files; each compiles in isolation
- Hold out one whole statement at its original site
- Build G_Γ from context Γ

Whole-statement measures

Precision reference-compiler acceptance rate of token-bounded uniform samples from $\mathcal{L}(G_\Gamma)$.

Recall fraction of whole statements admitted after fresh-name/literal normalization.

Name and literal normalization

Held-out statement x^*

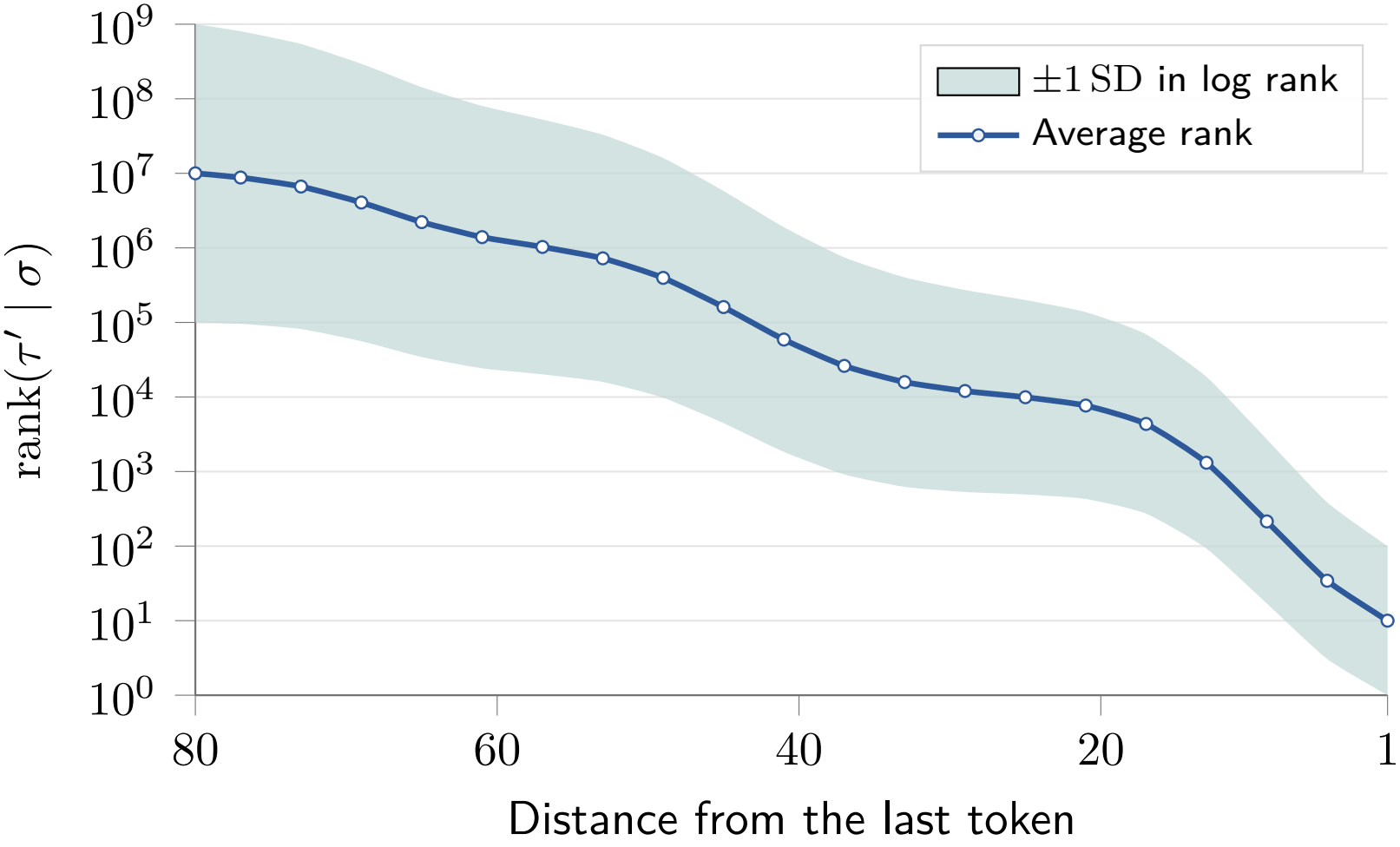
```
for (const auto& animal : animals)
  introduce(*animal, 2);
```

Normalized form admitted by G_Γ

```
for (const auto& {fresh_id} : animals)
  introduce(*{fresh_id}, 0);
```

Fresh-name handling (capture-avoiding). Only a binder introduced by the held-out statement, fresh relative to Γ, and all its bound uses may map to `{fresh_id}`. Keywords and contextual/API names stay exact; literals canonicalize to `0` or `""`.

Evaluation: Mock Results



At each prefix cutoff of a statement $\sigma\tau'$, σ is observed and τ' held out; the curve shows mean rank and variance. Registered-report target; simulated values only.

Future work

Scale up & validate

- Move from standalone files to full APIs and repository-scale corpora
- Test samples, report coverage, precision/recall, time, and memory

Increase expressivity

- Add fields, operators, conversions, variadics, and higher arities
- Target more languages; explore DCGs and richer type constraints

Smarter generation

- Materialize only productions reachable from the current prefix; quotient symmetric states
- Among CFG-admitted completions, use corpus or model scores to put likely continuations first

Performance optimizations

- **Lazy construction:** same bounded language, less memory?
- **Reranking:** same valid candidates, lower held-out rank?